Unidade 01 | Capítulo 05

Criando uma Aplicação SaaS

Prof.: Allberson Dantas





Sumário

- Estrutura do Projeto;
- Definição de Planos (freemium, premium);
- Implementação Básica de Autenticação;
- Banco de Dados Multitenant;
- Teste usando PostMan.

Visão Geral da Aula

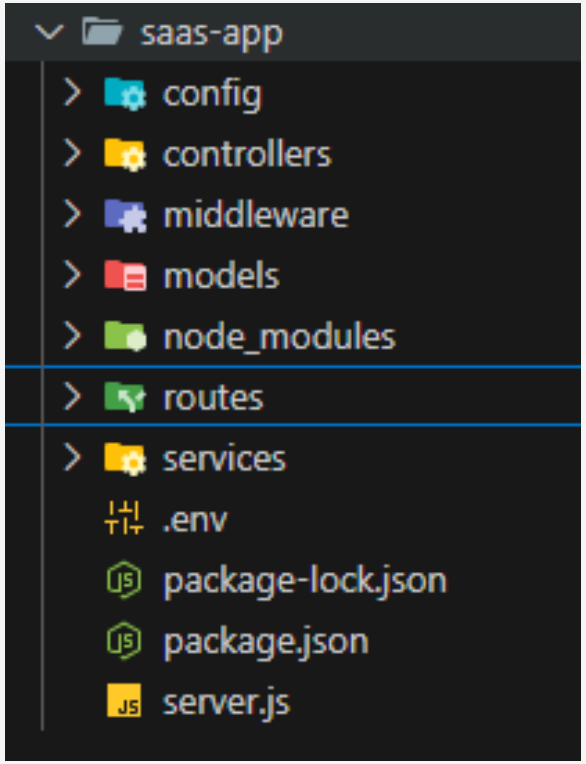
Esta aula orienta o aluno na construção de uma aplicação SaaS básica, com:

- Arquitetura modular e escalável;
- Cadastro e autenticação de usuários;
- Suporte a múltiplos clientes (tenants);
- Plano gratuito com controle de acesso.

A aplicação será um serviço genérico que permite aos clientes (empresas) gerenciar seus próprios usuários e dados

Estrutura do Projeto

O projeto será dividido em camadas para facilitar manutenção e escalabilidade:



Configurações do Projeto

Para rodar o projeto, é necessário realizar algumas configurações iniciais. Todas essas etapas serão apresentadas no próximo material.

Abra o arquivo e siga o passo-a-passo da Seção 1 - Configurações do Projeto.

Inicialização da Aplicação

```
require('dotenv').config();
const express = require('express');
const mongoose = require('./config/db');
const authRoutes = require('./routes/auth');
const userRoutes = require('./routes/user');
const tenantRoutes = require('./routes/tenant');

const app = express();
app.use(express.json());

app.use('/auth', authRoutes);
app.use('/users', userRoutes);
app.use('/tenants', tenantRoutes);

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`Servidor rodando na porta ${PORT}`));
```

No arquivo server.js nós inicializamos nossa aplicação:

server.js:

- Carrega .env e configura o servidor Express;
- Conecta com MongoDB via módulo externo;
- Define rotas: /auth, /users, /tenants;
- Inicia o servidor e escuta na porta configurada.

Conexão com o MongoDB

```
const mongoose = require('mongoose');

mongoose.connect(process.env.MONGO_URI, {
  useNewUrlParser: true,
  useUnifiedTopology: true
}).then(() => console.log('MongoDB conectado'))
  .catch(err => console.log('Erro de conexão:', err));

module.exports = mongoose;
```

Este módulo é responsável por estabelecer a conexão da aplicação com o banco de dados MongoDB:

- Usa o mongoose para conectar com o MongoDB;
- Lê a string de conexão do .env (MONGO_URI);
- Aplica configurações modernas da conexão;
- Exibe mensagens de sucesso ou erro;
- Exporta o mongoose para uso em outros módulos.

Modelo User

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  name: String,
  email: String,
  password: String,
  tenantId: String,
});

module.exports = mongoose.model('User', userSchema);
```

O modelo User (models/User.js) define a estrutura dos usuários da aplicação. Principais campos:

- name: Nome do usuário;
- email: E-mail do usuário;
- password: Senha (armazenada com hash);
- tenantId: Identificador do locatário ao qual o usuário pertence.

Modelo Tenant

```
const mongoose = require('mongoose');

const tenantSchema = new mongoose.Schema({
  name: String,
  plan: String // freemium ou premium
});

module.exports = mongoose.model('Tenant', tenantSchema);
```

O modelo Tenant (models/Tenant.js) representa um cliente ou organização que usa a aplicação SaaS. Ele define a separação dos dados por empresa.

Principais campos:

- name: Nome do locatário (ex: empresa);
- plan: Tipo de plano contratado (freemium ou premium).

Modelo Plan

```
const planSchema = new mongoose.Schema({
  name: String,
  maxUsers: Number,
  price: Number
});

module.exports = mongoose.model('Plan', planSchema);
```

O modelo Plan define os tipos de planos oferecidos pelo SaaS. Cada plano controla o limite de usuários e o valor cobrado. Principais campos:

- name: Nome do plano (ex: “freemium”, “premium”);
- maxUsers: Quantidade máxima de usuários;
- permitidosprice: Preço do plano.

Controlador de Autenticação

```
const jwt = require('jsonwebtoken');
const bcrypt = require('bcryptjs');
const User = require('../models/User');

exports.login = async (req, res) => {
  const { email, password } = req.body;
  const user = await User.findOne({ email });
  if (!user || !bcrypt.compareSync(password, user.password))
    return res.status(401).send('Credenciais inválidas');

  const token = jwt.sign(
    { userId: user._id, tenantId: user.tenantId },
    process.env.JWT_SECRET,
    { expiresIn: '1h' }
  );

  res.json({ token });
};

exports.register = async (req, res) => {
  const { name, email, password, tenantId } = req.body;
  const hashed = bcrypt.hashSync(password, 8);
  const user = await User.create({ name, email, password: hashed, tenantId });
  res.status(201).json(user);
};
```

Este arquivo (controllers/authController.js) centraliza a lógica de autenticação da aplicação SaaS, utilizando JWT (JSON Web Token) e bcrypt para criptografia de senhas.

Tecnologias utilizadas:

- jsonwebtoken para geração de token;
- bcryptjs para criptografia de senha;
- mongoose para interação com o banco de dados.

Rotas de Autenticação

```
const express = require('express');
const { login, register } = require('../controllers/authController');
const router = express.Router();

router.post('/login', login);
router.post('/register', register);

module.exports = router;
```

Routes/auth.js define os endpoints públicos relacionados à autenticação de usuários, utilizando o Express Router.

Rotas disponíveis:

- POST /auth/login: Autentica o usuário e retorna um token JWT.
- POST /auth/register: Registra um novo usuário no sistema.

Middleware de Autenticação

```
const jwt = require('jsonwebtoken');

module.exports = (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(403).send('Token ausente');
  jwt.verify(token, process.env.JWT_SECRET, (err, decoded) => {
    if (err) return res.status(401).send('Token inválido');
    req.user = decoded;
    next();
  });
};
```

O middleware (middleware/verifyToken.js) garante que apenas usuários autenticados acessem rotas protegidas na aplicação SaaS.

Como funciona:

- Extrai o token JWT do cabeçalho Authorization (Bearer <token>).
- Verifica a validade do token usando o JWT_SECRET definido no .env.
- Se válido, anexa os dados do usuário (req.user) à requisição.
- Se inválido ou ausente, bloqueia o acesso com os códigos de erro apropriados (403 ou 401).

Controller de Usuários

Este controlador (controllers/userController.js) gerencia a criação de novos usuários em um ambiente multi-tenant com restrições baseadas no plano do locatário.

```
const User = require('../models/User');
const Tenant = require('../models/Tenant');

exports.addUser = async (req, res) => {
  const { name, email, password } = req.body;
  const tenantId = req.user.tenantId;

  const tenant = await Tenant.findById(tenantId);
  const currentUsers = await User.countDocuments({ tenantId });

  const planLimits = tenant.plan === 'freemium' ? 3 : Infinity;
  if (currentUsers >= planLimits)
    return res.status(403).send('Limite de usuários atingido para este plano');

  const hashed = require('bcryptjs').hashSync(password, 8);
  const newUser = await User.create({ name, email, password: hashed, tenantId });
  res.status(201).json(newUser);
};
```

Rotas de Usuários

Routes/user.js define a rota protegida para criação de novos usuários, utilizando o middleware de autenticação e o controlador correspondente.

```
const express = require('express');
const { addUser } = require('../controllers/userController');
const verifyToken = require('../middleware/verifyToken');
const router = express.Router();

router.post('/', verifyToken, addUser);

module.exports = router;
```

Cadastro de Tenant

Routes/tenant.js define a rota responsável por registrar **novos tenants** (clientes ou empresas) na aplicação SaaS.

```
const express = require('express');
const Tenant = require('../models/Tenant');
const router = express.Router();

router.post('/', async (req, res) => {
  const { name, plan } = req.body;
  const tenant = await Tenant.create({ name, plan });
  res.status(201).json(tenant);
});

module.exports = router;
```

Rodando o Projeto

Para executar o projeto corretamente, siga o passo a passo que será disponibilizado na sequência deste conteúdo.

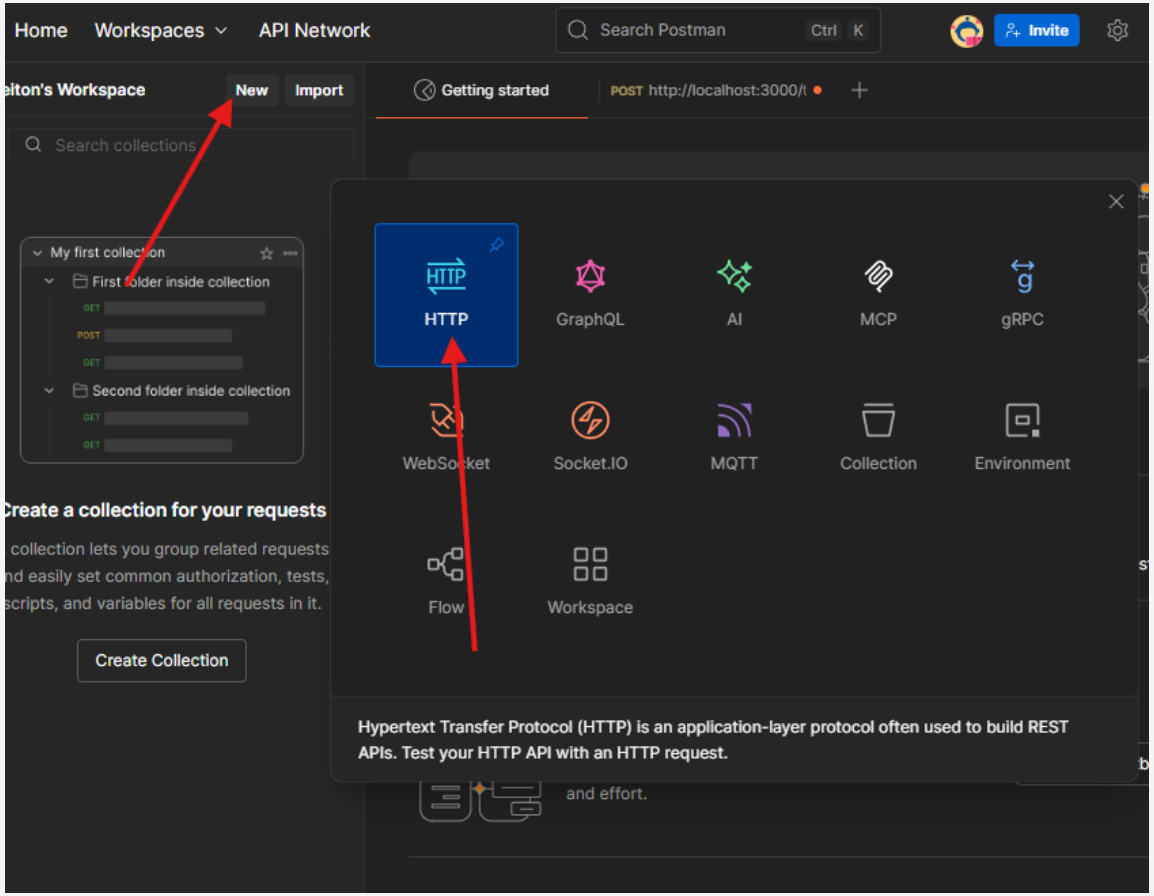
Após instalar o PostMan, vamos testar a aplicação.

Rodando o Projeto

O Postman é uma ferramenta poderosa para testar APIs de forma prática e visual. Aqui vai um guia rápido para você começar a usar:

Criar uma requisição:

Abra o Postman e clique em “New” → “Request”

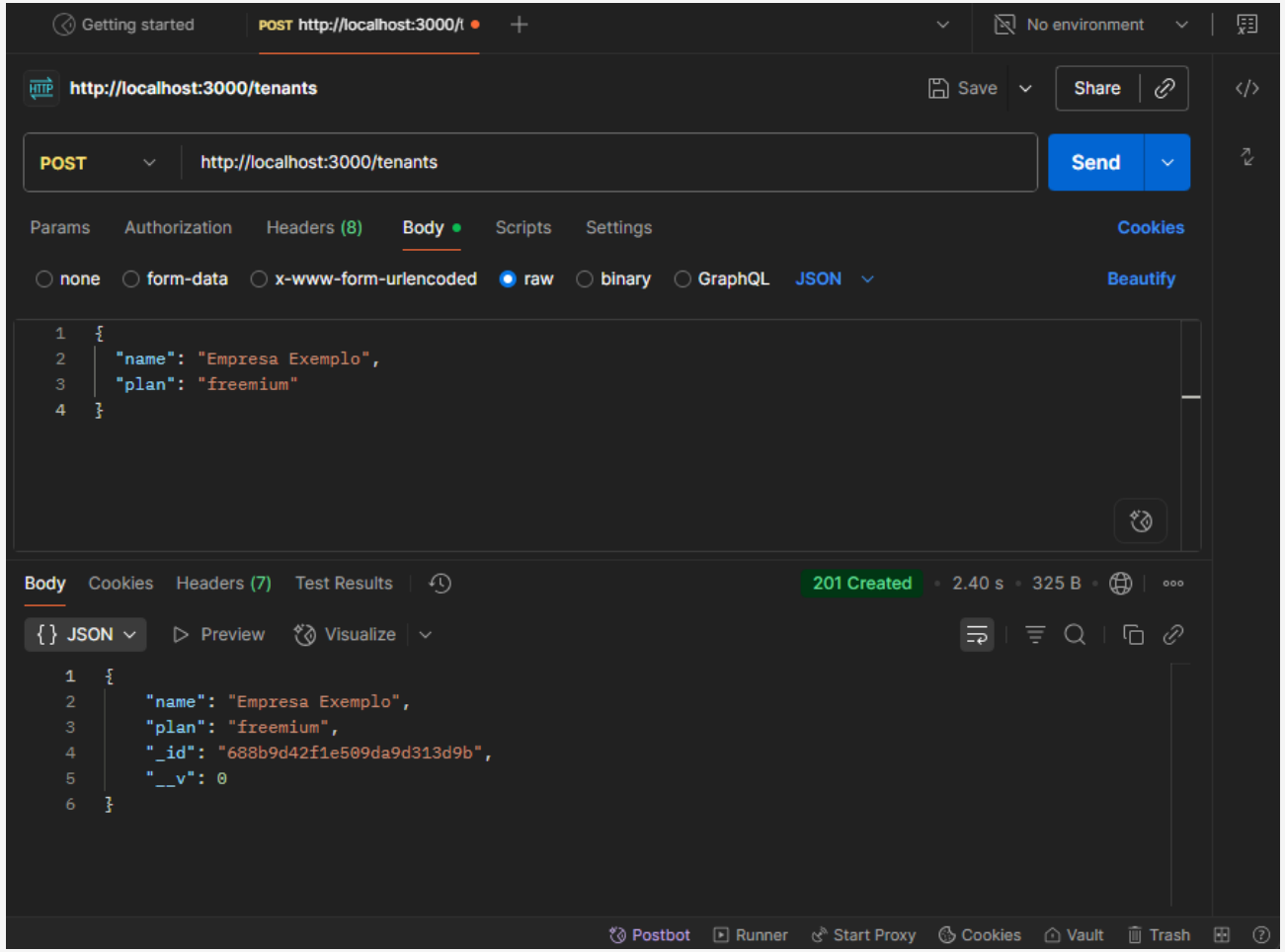


Rodando o Projeto

Escolha o método HTTP (ex: POST, GET, PUT, DELETE);

Insira a URL da sua API (ex: http://localhost:3000/tenants);

Vá na aba “Body” e selecione “raw” → “JSON” e insira o corpo da requisição.





Lista de Exercícios e Tarefas

Responda o Quiz do Capítulo.

Desenvolva as questões da TAREFA 21 – Criando uma Aplicação SaaS.

CAPACITA iRede

INOVAÇÃO E TECNOLOGIA

RESIDÊNCIA EM TIC 20



EXECUTOR

iRede INOVAÇÃO E TECNOLOGIA

INICIATIVA

MCTI FUTURO

COORDENADORA PPI

Softex

MINISTÉRIO DA CIÊNCIA, TECNOLOGIA E INOVAÇÃO

GOVERNO FEDERAL BRASIL UNIÃO E RECONSTRUÇÃO

*Este projeto foi apoiado pelo Ministério da Ciência, Tecnologia e Inovações, com recursos da Lei nº 8.248, de 23 de outubro de 1991, no âmbito do PPI-SOFTEX, coordenado pela Softex e publicado Residência em TIC 20, DOU 01245.002631/2023-58.